

Perceptron Implementation

```
In [1]: input_data = [100, 200, 300]
```

```
In [2]: weights = [3,4,5]
```

```
In [17]: bias = -29000
```

```
In [18]: learning_rate = 0.01
```

```
In [19]: target = 1
```

compute weighted sum

```
In [20]: z = weights[0]*input_data[0] + weights[1]*input_data[1] + weights[2]
```

activation function

```
In [21]: if z >= 0 :  
        y_pred = 1  
else:  
    y_pred=0
```

```
In [22]: error = target - y_pred  
print(error)
```

0

```
In [23]: if (error == 0):  
        print ("Predicted and actual values are equal, hence no need of  
else:  
    for i in range(len(weights)):  
        weights[i] = weights[i] + learning_rate * error * input_dat  
        print(weights[i])  
    bias = bias + learning_rate * error  
    print(bias)
```

Predicted and actual values are equal, hence no need of parameter up
dation

conclusion

```
In [24]: print("Target : ",target, " Predicted value: ", y_pred)  
print("Error: ", error)  
print("Updated weights: ", weights)  
print("Updated Bias: ", bias)
```

Target : 1 Predicted value: 1
Error: 0
Updated weights: [330.0, 640.0, 500.0]
Updated Bias: -29000

backpropagation

```
In [25]: weights = [330, 640, 500]  
        bias = -123090
```

```
In [26]: z = weights[0]*input_data[0] + weights[1]*input_data[1] + weights[2]
```

```
In [27]: if z >= 0 :  
        y_pred = 1  
        else:  
        y_pred=0
```

```
In [28]: error = target - y_pred  
        print(error)
```

0

```
In [29]: if (error == 0):  
        print ("Predicted and actual values are equal, hence no need of  
        else:  
        for i in range(len(weights)):  
            weights[i] = weights[i] + learning_rate * error * input_dat  
            print(weights[i])  
            bias = bias + learning_rate * error  
            print(bias)
```

Predicted and actual values are equal, hence no need of parameter up
dation

```
In [ ]:
```